

Recall that we can model the set of terms in the (pure) λ -calculus in OCaml as a data type

```
type lam = V of string | Abs of string * lam | App of lam * lam;;
```

DeBruijn Indexed terms are an alternative syntax for the λ -calculus, where there are no binding occurrences and corresponding bound occurrences. The advantage of this notation is that we do not have to grapple with bureaucratic and boring notions such as α -conversion. (deBruijn is pronounced approximately as “de-brine”.)

The intuition is that variables act as place holders that mark and link occurrences of the parameter variable (bound occurrences) to their binding occurrence. Instead of bound variables, we count the number of λ -bindings — on the path from a variable occurrence to the root of the term — that are (strictly) crossed in order to reach the nearest λ -binding occurrence for that particular variable. In the deBruijn notation, we suppress the binding occurrence, and replace the bound occurrence by an index, namely this number. If in reaching the binding occurrence, no λ -bindings with other variables are crossed, then the bound occurrences are immediately in the scope of the binder (local variables), and so have index 0. If there isn’t any binding occurrence for a particular variable occurrence on the path to the root, then that variable occurrence is *free*. So the only variables that appear in a deBruijn term are free variables.

We create in OCaml an abstract syntax for deBruijn terms, which closely parallels that for the λ -calculus.

```
(* Debruijn indices *)  
type deb = Vd of string | Index of int  
          | Absd of deb | Appd of deb * deb;;
```

Note that we have introduced indexes which are integers ≥ 0 , i.e., natural numbers. Note also that abstractions, namely Absd terms, do not have a binding variable occurrence.

There isn’t a 1-1 correspondence between indices (plural of index) and bound variable occurrences. One can easily see that two different occurrences of the same bound variable x are given different indices, and the same index, e.g. 0, in different sub-terms can correspond to different variables. For example the term $(\lambda x. (x (\lambda y. x))) (\lambda z. z)$ is rendered in deBruijn terms as $(\lambda. (\underline{0} (\lambda. \underline{1}))) (\lambda. \underline{0})$. Note that the two different bound occurrence of the same variable x are given indices 0 and 1, whereas the two different index occurrences of 0 correspond to different variables, x and z respectively. (We have deliberately underlined the indices to emphasise that they are not intended to be numerals).

Exercise. Render a set of λ -calculus terms, e.g., the Church numerals in the OCaml syntax for deBruijn terms.

The conversion of λ -calculus terms to deBruijn indexed terms is straightforward. However, while the definition asks us to consider the path from each variable occurrence, at a leaf, to the root, i.e., bottom up, this is a painful and inefficient process. Instead there

is a simple top-down technique that stacks all the variables with a λ -binding occurrence on a downward sweep of the abstract syntax tree, placing the most recently encountered binding variable occurrence at the top of the stack.

The auxiliary function *pos* counts the distance from the top of the stack of a particular variable, raising an exception if it is not present in the stack.

```
exception NotIn;;

let rec pos x l = match l with
  [ ] -> raise NotIn
  | y::ys -> if x=y then 0 else 1+(pos x ys)
;;
```

The translation of λ -calculus terms into deBruijn terms is coded as the recursive function *deB*: *lam* -> *deb*. Variables (which are bound variables) are translated to indices if they have binding occurrences, and otherwise are left unchanged (except for the change in the constructor name from *v* to *v_d* necessitated by OCaml not allowing constructors to be shared between different data types). Note the use of the **try-with** OCaml construct, which allows us to raise and field exceptions. An abstraction (a λ -calculus term with constructor *Abs*) remains an abstraction (but now with constructor *Abs_d*), but strip away the binding occurrence variable, and stack it onto the list of variables whose binder has been encountered on the path from the root. Applications (λ -calculus terms with constructor *App*) simply involve recursion in both sub-terms (and the constructor is now rendered as *App_d*).

```
let rec deB e s = match e with
  V x -> (try Index(pos x s) with NotIn -> Vd(x))
  | Abs(x,e1) -> Absd( deB e1 (x::s) )
  | App(e1,e2) -> Appd( (deB e1 s), (deB e2 s))
;;
```

However, not all operations are easy to implement in deBruijn terms. One of the bread-and-butter operations of the λ -calculus, name substitution requires considerable adjustment of indices. For this we use an auxiliary function *shift* that takes a cut-off index *c* (the lowest index given to a relatively-speaking global variable) and an offset *d*, and shifts all indices in a deBruijn term *e* that are greater or equal to the cut-off *c* by adding *d* to these indices. Note that inside an *Abs_d* term, the cut-off is increased by 1.

```
(* cut-off c [lowest index of a relative global], shift by d above
this...*)
```

```
let rec shift c d e = match e with
  Vd(x) -> Vd(x) (* free vbl stays unchanged *)
  | Index(i) -> if i < c then Index(i) (* below cutoff: unchanged *)
  else Index(i+d) (* above cutoff: shifted by d *)
  | Absd(e1) -> Absd( shift (c+1) d e1) (* absd: cutoff incremented *)
  | Appd(e1,e2) -> Appd(shift c d e1, shift c d e2) (* recur on both *)
;;
```

Substitution now has to be formulated too, stated in terms of replacing, in a term e , occurrences of index j by a term e' . Note that the substitution here operates by replacing an index — i.e., a bound occurrence of a variable — in particular, all bound occurrences of what was marked as the parameter x in the λ -calculus. (Now recall that different bound occurrences of the same variable tied to the same binding occurrence may have different indices depending on where they occur in the deBruijn term. Also observe that while there is no worry about capturing free variables during the substitution in deBruijn terms, there is a complication — that the indices within the term e' may have to be adjusted depending on where the term e' is being pasted in).

The substitution operation therefore uses the auxiliary function *shift* to adjust all indices within the term e' (the cut-off is 0 and the shift is +1), and incrementing the target index j by 1 for each *Absd* encountered, when pasting in a term below a binder. Recall that under the *Absd* binder, index $j+1$ corresponds to the same binding variable given index j just outside the *Absd*.

```
(* Substitution *)
```

```
let rec substdeB e e' j = match e with
  | Vd(x) -> Vd(x)
  | Index(i) -> if i=j then e' else Index(i)
  | Absd(e1) -> Absd( substdeB e1 (shift 0 1 e') (j+1) )
  | Appd(e1, e2) -> Appd(substdeB e1 e' j, substdeB e1 e' j)
;;
```

Execution in the λ -calculus was expressed using β -reduction. We present a partial version of weak normal form (call-by-name) reduction. The important case is the 1-step β -reduction, which in deBruijn terms is written as $((\lambda . e_1) e_2)$. In our OCaml encoding this is rendered as `Appd(Absd(e1), e2)`.

The 1-step β -reduction of $(\lambda x . e_1) e_2$ to the term corresponding to $e_1[e_2/x]$ in the absence of explicitly mentioning formal parameter x is achieved by taking the representation of e_1 and substituting a version of the representation of e_2 (that has all indices upped by 1) for the the immediate parameter, which is denoted by index 0. Then since the λ -binder has now been removed, we downshift all indices by 1 (i.e., add -1 to each index).

Thus $e_1[e_2/x]$ is rendered in our OCaml representation as:

```
(shift 0 (-1) (substdeB e1 (shift 0 1 e2) 0))
```

We now present the multi-step β -reduction to weak normal form as follows:

```
(* beta reduction – multi-step to weak normal form *)
```

```
let rec reduce e = match e with
  | Vd(x) -> Vd(x) (* wnf *)
  | Index(i) -> raise NotIn (* error *)
  | Absd(e1) -> Absd(e1) (* wnf *)
  | Appd(Absd(e1), e2) ->
    (let c1 = (shift 0 (-1) (substdeB e1 (shift 0 1 e2) 0))
     in reduce c1) (* multiple steps *)
```

```
| Appd(Vd(x), e2) -> Appd( Vd(x), reduce e2)
| Appd(e1, e2) -> let wnf1 = reduce e1
                   in reduce (Appd(wnf1, e2))
;;
```

Exercise: Define a translation `lamify: deb -> lam` which preserves the intended program behaviour. What should happen if you compose this program with `deB`?